Health care analysis Project

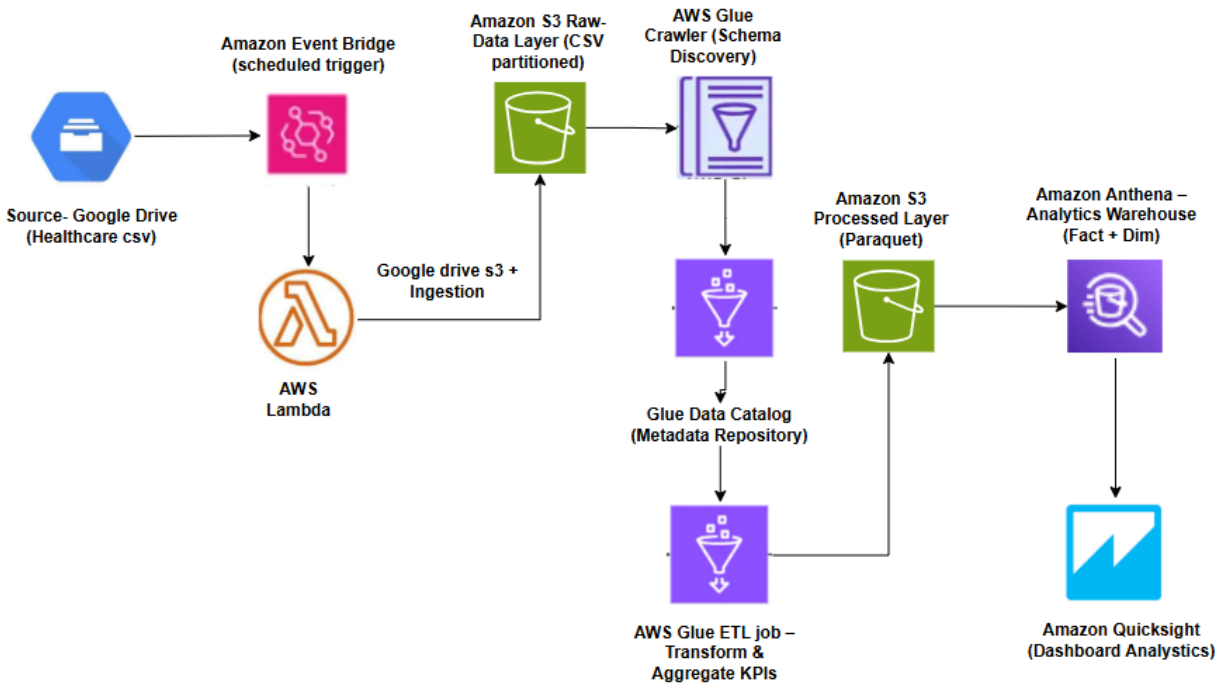
1. Project Overview-

This project implements an end-to-end AWS-based data lake and analytics platform designed to analyze healthcare staffing and operational performance across facilities. The primary objective of the solution is to provide a unified and centralized view of nurse availability, staffing workload, occupancy levels, and overall facility performance.

The platform enables the calculation of key performance indicators such as nurse-to-patient ratios, total staffing hours, overtime trends, and bed utilization rates. These insights support data-driven workforce planning, operational efficiency improvements, and better patient care outcomes.

2. Project Architecture-

The solution follows a modern data lake and analytics warehouse architecture built entirely on AWS services. The workflow begins with data ingestion from Google Drive and progresses through automated orchestration, transformation, warehousing, and visualization.



3. Data Ingestion and Orchestration

The source datasets, including PBJ Daily Nurse Staffing data and CMS Nursing Home supporting files, are stored in Google Drive. These files serve as the raw input for the analytics pipeline.

Amazon EventBridge is configured with a scheduled rule that automatically triggers the ingestion process on a defined frequency, such as daily or weekly. This ensures that the system operates in a fully automated and production-ready manner without manual intervention.

When triggered, AWS Lambda connects securely to Google Drive using API authentication, downloads the latest CSV files, and uploads them into the Amazon S3 Raw Landing Layer. This serverless ingestion mechanism eliminates the need for infrastructure management while maintaining scalability and reliability.

4. Raw Data Lake Layer

The Amazon S3 Raw Landing Layer stores the data exactly as it is received from the source system. No transformations are applied at this stage. This approach preserves data integrity, enables traceability, and allows historical replay if needed. The raw layer functions as the foundational storage tier of the data lake architecture.

5. Metadata Management

An AWS Glue Crawler scans the raw data stored in Amazon S3 and automatically infers the schema of the datasets. The crawler registers the discovered tables in the AWS Glue Data Catalog, making the data queryable and accessible for transformation jobs.

This automated schema discovery simplifies metadata management and ensures that structural changes in source files can be handled efficiently.

6. Data Transformation and KPI Aggregation

AWS Glue ETL jobs perform the core transformation logic of the platform. The ETL process cleans and standardizes the raw datasets, joins staffing information with facility-level metadata, and computes business-critical key performance indicators.

The transformation layer calculates metrics such as nurse-to-patient ratios, total nurse hours by facility and state, overtime percentages, and bed utilization rates. The processed data is written back to Amazon S3 in an optimized Parquet format, partitioned for improved query performance.

This step converts raw operational data into analytics-ready datasets.

7. Processed Data Layer

The Amazon S3 Processed Layer stores the curated and aggregated datasets generated by the ETL process. By storing the data in columnar format and using partitioning strategies, the system ensures efficient downstream analytical querying.

This layer serves as the clean and structured data foundation for the analytics warehouse.

8. Analytics Warehouse

Amazon Anthena functions as the centralized analytics warehouse. The curated datasets are loaded into Anthena and modeled using a dimensional design approach consisting of fact and dimension tables.

Fact tables store measurable staffing and occupancy metrics, while dimension tables store descriptive attributes such as facility, state, and date. This star schema design enables high-performance analytical queries and supports scalable reporting requirements.

9. Visualization and Reporting

Amazon QuickSight connects directly to the Anthena analytics warehouse to build interactive dashboards. The dashboards provide visual insights into staffing trends, nurse-to-patient ratios, occupancy rates, and facility-level comparisons.

The reporting layer enables management teams to identify understaffed facilities, monitor overtime trends, compare staffing against patient load, and make informed workforce planning decisions.

10. Business Value

This solution provides a unified view of healthcare staffing operations across facilities. It helps identify workforce imbalances, monitor operational efficiency, and analyze the relationship between staffing levels and patient capacity. By leveraging a scalable AWS-native architecture, the platform ensures automation, reliability, and performance while delivering actionable operational insights.

Solution Design Document

Healthcare Staffing Analytics Platform (AWS Data Lake Architecture

1. Executive Summary

This document describes the design and implementation of a cloud-based Healthcare Staffing Analytics Platform built on Amazon Web Services (AWS). The solution ingests staffing and facility datasets from Google Drive, processes and transforms the data using AWS Glue, stores curated datasets in Amazon S3, loads them into Amazon Anthena for analytics, and presents insights through Amazon QuickSight dashboards.

The system provides operational visibility into nurse-to-patient ratios, staffing workload, occupancy levels, and facility performance metrics to support workforce planning and decision-making.

2. Business Objectives

The primary business objectives of the solution are:

- Provide a unified view of staffing performance across facilities.
- Calculate nurse-to-patient ratios and staffing workload metrics.
- Analyze occupancy and bed utilization trends.
- Enable leadership to identify understaffed facilities.
- Automate data ingestion and reporting processes.
- Deliver interactive dashboards for operational decision-making.

3. Functional Requirements

The system must:

1. Ingest staffing and facility datasets from Google Drive on a scheduled basis.
2. Store raw data in a secure and scalable data lake.
3. Automatically detect schema and manage metadata.
4. Transform raw data into analytics-ready datasets.
5. Calculate key staffing and facility KPIs.
6. Load curated data into an analytics warehouse.
7. Provide interactive dashboards with filtering and drill-down capabilities.
8. Support historical trend analysis.

4. Non-Functional Requirements

- The system must be fully automated and require minimal manual intervention.
- The solution must be scalable to accommodate growing datasets.
- Data storage must be durable and secure.
- The platform must provide high availability.
- Query performance must support interactive dashboard use.
- Access must be controlled through IAM roles and permissions.

5. High-Level Architecture

The solution follows a modern data lake and warehouse pattern. Please find below diagram for high-level architecture diagram-

Google Drive
→ Amazon EventBridge (Scheduled Trigger)
→ AWS Lambda (Ingestion)
→ Amazon S3 Raw Landing Layer
→ AWS Glue Crawler (Metadata Management)
→ AWS Glue ETL (Transform & KPI Aggregation)
→ Amazon S3 Processed Layer
→ Amazon Anthena
→ Amazon QuickSight (Dashboard & Reporting)

6. Detailed Component Design

6.1 Source Layer – Google Drive

The source system contains:

- PBJ Daily Nurse Staffing data
- CMS Nursing Home supporting CSV files

These files are periodically updated and serve as the raw data source.

6.2 Orchestration – Amazon EventBridge

Amazon EventBridge is configured with a scheduled rule that triggers the ingestion workflow at a defined frequency (daily/weekly). This ensures automated data refresh without manual intervention.

6.3 Ingestion Layer – AWS Lambda

AWS Lambda performs the following:

- Connects securely to Google Drive via API.
- Downloads the latest CSV files.
- Uploads files into Amazon S3 Raw Layer.
- Logs execution metrics to CloudWatch.

Lambda is chosen for its serverless architecture, scalability, and cost efficiency.

6.4 Data Lake – Amazon S3

Raw Layer

The Raw Landing Layer stores data exactly as received from the source. It ensures traceability and supports reprocessing if needed.

Processed Layer

The Processed Layer stores transformed datasets in Parquet format. Data is partitioned by attributes such as state and month to optimize performance.

6.5 Metadata Management – AWS Glue Crawler

The Glue Crawler scans S3 Raw data, infers schema automatically, and updates the Glue Data Catalog. This allows Glue ETL jobs and other AWS services to reference structured tables.

6.6 Transformation Layer – AWS Glue ETL

The Glue ETL job performs:

- Data cleansing and standardization.
- Joining staffing and facility datasets.
- KPI calculations, including:
    - Nurse-to-patient ratio
    - Total nurse hours
    - Overtime percentage
    - Occupancy rate
- Writing processed data to S3 in optimized format.

The ETL logic is implemented using PySpark within AWS Glue.

6.7 Analytics Warehouse – Amazon Anthena

Amazon Anthena serves as the centralized analytics warehouse.

A dimensional model is implemented:

- Fact Tables:
    - Staffing Metrics
    - Facility Performance Metrics
- Dimension Tables:
    - Facility
    - State
    - Date

This star schema design ensures efficient analytical queries and supports scalable reporting.

6.8 Visualization – Amazon QuickSight

Amazon QuickSight connects to Amazon Anthena to build interactive dashboards. The dashboards provide:

- Staffing trends over time
- Nurse-to-patient ratios by facility
- Occupancy and bed utilization rates
- State-level performance comparisons

The visualization layer enables drill-down and filtering capabilities.

7. Data Model Overview

The data model follows a star schema:

Fact_Staffing

- Facility_ID
- Date_ID
- Total_Nurse_Hours
- Overtime_Hours
- Nurse_Patient_Ratio

Dim_Facility

- Facility_ID
- Facility_Name
- State
- Certified_Beds

Dim_Date

- Date_ID
- Month
- Year

This model supports fast aggregations and time-series analysis.

8. Security Design

- IAM roles restrict access to S3, Glue, Anthena, and QuickSight.
- S3 buckets are configured with encryption at rest.
- Anthena uses role-based access control.
- Lambda execution roles follow least-privilege principle.
- CloudWatch monitors execution logs.

9. Scalability & Performance

- Amazon S3 provides virtually unlimited storage scalability.
- Glue ETL uses distributed Spark processing.
- Data is stored in Parquet format for optimized queries.
- Anthena enables columnar storage and parallel processing.
- Partitioning strategy improves query efficiency.

10. Monitoring & Logging

- AWS Lambda logs execution in CloudWatch.
- Glue job logs are monitored for failures.
- Anthena query performance is monitored via system tables.
- EventBridge ensures job scheduling reliability.

11. Benefits of the Solution

- Fully automated data pipeline, scalable and serverless ingestion and transformation.
- Optimized analytics performance.
- Centralized and consistent reporting.
- Actionable operational insights for workforce planning.

12. Future Enhancements

- Implement incremental data loading.

- Add Athena for ad-hoc S3 queries.
- Introduce CI/CD pipeline for infrastructure deployment.
- Add anomaly detection for staffing irregularities.
- Implement data quality validation framework.

Conclusion

This Healthcare Staffing Analytics Platform leverages AWS cloud-native services to deliver a scalable, automated, and high-performance analytics solution. The architecture follows best practices in data lake design, metadata management, dimensional modeling, and business intelligence reporting.

The system enables healthcare leadership to gain meaningful insights into staffing patterns, operational efficiency, and facility performance, ultimately supporting improved decision-making and patient care outcomes.

Design Decisions & Trade-offs

This solution was designed using AWS-managed services to minimize operational overhead while still supporting scalable ingestion, transformation, and analytics. Several design decisions were made based on cost, maintainability, performance, and the type of workload (batch analytics).

1. Why EventBridge + Lambda for ingestion
Amazon EventBridge was selected to trigger ingestion on a predictable schedule because the source data is delivered as batch CSV extracts rather than continuous streams. AWS Lambda was chosen for ingestion because it provides a serverless and scalable way to download files from Google Drive and land them in Amazon S3 without managing any servers. The trade-off is that Lambda has execution time and memory limits, so for very large files or long ingestion workflows, an ECS/Fargate job or AWS Batch would be more suitable.

2. Why Amazon S3 for Raw and Processed zones
Amazon S3 was used as the data lake storage layer because it is durable, cost-effective, and scales automatically. Separating the data into a Raw Landing Layer and a Processed Layer ensures traceability and supports reprocessing if transformation logic changes. The trade-off is that S3 is not a database, so interactive query performance depends on downstream services (such as Anthena or Athena) and on how well the data is partitioned and stored.

3. Why Glue Crawler and Glue Data Catalog
AWS Glue Crawler was used to automatically infer schemas and register metadata in the Glue Data Catalog, which reduces manual schema management and makes the pipeline easier to maintain. The trade-off is that crawler-based inference can sometimes infer incorrect data types if the source files have inconsistent formatting, so production systems often add schema enforcement or validation rules.

4. Why AWS Glue ETL for transformations
AWS Glue ETL was selected because it provides serverless Spark processing that can scale for large batch workloads while integrating naturally with S3 and the Glue Data Catalog. Glue also supports writing optimized formats like Parquet and enables partitioning for faster downstream reads. The trade-off is that Glue jobs can have higher startup latency than lightweight Python scripts, and debugging can be slower compared to local development.

5. Why Parquet + partitioning in the Processed layer
Processed datasets are written in Parquet format because it is columnar, compressed, and significantly improves analytical query performance compared to CSV. Partitioning by fields such as state and month helps reduce scan cost and improves performance for dashboard queries. The trade-off is that partitioning too aggressively can create many small files, which can reduce performance and increase metadata overhead.

7. Why star schema (Fact + Dimensions) instead of one flat table
A star schema was implemented because it separates measurable metrics (facts) from descriptive attributes (dimensions), making the model easier to extend and improving query performance for reporting. It also aligns with how BI tools like QuickSight typically query data. The trade-off is that dimensional modeling requires upfront design effort and a clearer understanding of business entities and keys.

8. Why QuickSight for dashboards
Amazon QuickSight was selected because it integrates directly with Anthena, supports interactive filtering and drilldowns, and avoids the need to host and manage a custom web application. It also makes the solution fully AWS-native. The trade-off is reduced UI customization compared to Streamlit, and some advanced visuals or custom logic may be easier in a Python-based dashboard.

9. Data refresh and operational reliability considerations
The pipeline is batch-based and designed to refresh on a schedule, which is appropriate for periodic reporting and workforce planning. Near-real-time processing was intentionally avoided to reduce cost and complexity. The trade-off is that the dashboard is only as fresh as the latest scheduled load, but this is acceptable for staffing analytics where hourly latency is not typically required.

10. How will the Glue job be triggered?

In my design, the ingestion is scheduled using **EventBridge**, which triggers **Lambda** to pull files from Google Drive and land them in **S3 Raw**. After the upload completes, the pipeline triggers the **Glue workflow** (Crawler + ETL) in one of two ways:

- **Preferred (simple + reliable):** Lambda calls the AWS Glue API (`StartJobRun`) to start the Glue ETL job after it finishes uploading files. This gives end-to-end control and ensures ETL runs only when ingestion completes.
- **Alternative:** Use an **S3 ObjectCreated** event on the raw prefix to trigger an EventBridge rule that starts the Glue job. This works well if we land a single "_SUCCESS" marker file after all uploads.

So overall it's a **schedule-driven batch pipeline**: EventBridge → Lambda → S3 → Glue.

11. What happens if Lambda times out?

If Lambda times out mid-download/upload, the run is considered failed and the data could be partially loaded. To handle this cleanly, I designed the ingestion to be **idempotent** and resilient:

- Lambda writes data into a **run-specific folder** (for example `raw/<dataset>/load_date=YYYY-MM-DD/run_id=<uuid>/...`). That prevents overwriting good data with partial data.
- Lambda uploads a final **success marker file** (example: `raw/_manifest/load_date=YYYY-MM-DD/_SUCCESS.json`) only after all files are uploaded. Glue is triggered only when that success marker exists.
- If a timeout happens, the `_SUCCESS` marker is not written, so Glue won't run. The next scheduled run retries safely.
- If files are large, I can avoid timeouts by either (a) increasing Lambda timeout to max allowed, (b) downloading only incremental/new files, or (c) moving ingestion to **Step Functions + ECS/Fargate** for long-running downloads.

So the pipeline won't process partial loads, and retries are safe.

12. If the Glue job fails, will there be alerts?

Yes. Glue job failures are monitored using **EventBridge rules** and notifications are sent via **SNS** (or email/Slack integration).

- An EventBridge rule listens for Glue job state changes (FAILED / TIMEOUT).
- When a failure event occurs, it triggers an SNS topic.
- SNS sends alerts to subscribed endpoints (email, SMS, Slack via webhook/Lambda).

Additionally, Glue logs go to **CloudWatch Logs**, so I can quickly drill into the exact error and rerun the job.